CEH | Threat Analysis & Incident Response

Updated Academic Response & Evidence-Ready Report

Based on the six-page Intellipaat Capstone Project brief

Scope note: Use screenshots captured from the authorized lab environment for the final submission.

1. Executive Summary

The capstone covers cloud data protection, data classification, forensic evidence handling, PowerShell encoded-command analysis, process/memory injection, Sysinternals-based malware analysis, PE-file interpretation, malware incident notification, incident response, phishing defense, and a red-team workflow. The response is structured so that each exercise can be submitted as an auditable security-analysis report.

The source brief explicitly identifies four exercises plus three scenario-based questions and asks for screenshots/evidence where applicable. ṆfileciteÖturn1file0ÖL68-L94Ộ

2. Exercise 1 – Cloud Security, Forensics & Encoded PowerShell

2.1 Securing data in the cloud

1. Classify the data before migration. Identify confidentiality, integrity, availability, regulatory and business-criticality requirements.
2. Use encryption in transit with modern TLS and encryption at rest using the cloud provider's managed key-encryption capabilities or customer-managed keys where stronger control is required.
3. Apply least-privilege IAM, MFA, conditional access, separation of duties and workload/service identities.
4. Protect storage with private-by-default access, network restrictions, logging, monitoring, backup/versioning and data-loss-prevention controls.
5. Use tokenization or masking for sensitive fields where the application does not need the original value.
6. For highly sensitive information, combine encryption with strong key management, rotation, access reviews and centralized audit logging.

2.2 Public vs. private vs. hybrid cloud

The choice should be risk- and business-driven rather than based on a single universal preference.

Model	Security/operational characteristics	Suitable use cases
Public cloud	Provider-operated infrastructure, scalable services, shared-responsibility security model.	Elastic workloads, SaaS/PaaS, non-restricted or appropriately protected enterprise workloads.
Private cloud	Greater organizational control over infrastructure and security architecture; usually higher operational responsibility.	Highly controlled environments, specialized regulatory or legacy requirements.
Hybrid cloud	Combines environments and requires secure integration, identity federation, segmentation and consistent governance.	Organizations that need both controlled environments and public-cloud scalability.

For an enterprise with mixed regulatory, legacy and scalable workloads, a hybrid architecture can be considered when governance, identity, network segmentation and data-flow controls are mature.

2.3 Data classification

Classification	Example	Typical handling
Public	Published website content	Approved for public release; integrity controls still apply.
Internal	Routine internal procedures	Business use; authenticated access and controlled sharing.
Confidential	Business-sensitive reports, customer information	Need-to-know access, encryption, monitoring and controlled sharing.
Restricted / Highly Confidential	Credentials, cryptographic keys, highly sensitive personal/financial data	Strongest access control, encryption, privileged monitoring, DLP and strict retention/disposal.

Classification should be assigned by the data owner and reviewed when business use, regulatory obligations or sensitivity changes.

2.4 Forensic evidence – should the analyst write to the suspect hard drive?

No. The forensic analyst should not write investigative data to the original suspect drive because doing so can alter evidence, timestamps, metadata, deleted-file remnants or filesystem structures. The preferred approach is to acquire a forensic image using a validated process and write the image or analysis output to a separate, controlled evidence destination.

The requested analysis is consistent with forensic examination of potentially deleted/erased data and preservation of evidentiary integrity. The analyst should use write-blocking, hashing, chain-of-custody records and a documented evidence-handling process.

Evidence principle: preserve the original media, analyze a verified forensic copy, and document every handling step.

2.5 Encoded PowerShell analysis

The command uses PowerShell's encoded-command switch (-EncodedCommand / -EC). The blob is Base64-encoded PowerShell content represented as UTF-16LE text. The assignment asks the analyst to decode the blob and identify the URL, output filename, folder and ShellExecute method. ṆfileciteÖturn1file0ÖL22-L67Ộ

Decoded behavioral summary:

- The script creates a System.Net.WebClient instance and enumerates its methods.
- It looks for the WebClient DownloadData method.
- It constructs a URI and invokes the download operation.
- The returned bytes are written to a path derived from the CommonApplicationData folder.
- The decoded path uses a Windows executable filename and subsequently creates an object capable of invoking ShellExecute.
- The use of -NoP, -Exec Bypass and -EC is a notable defensive indicator because it combines PowerShell non-interactive execution, execution-policy bypass and encoded content.

Decoded indicators requested by the assignment

Question	Answer / analyst note
Encoding mechanism	Base64 encoding of UTF-16LE PowerShell text, supplied through PowerShell -EC / -EncodedCommand.
URL	The encoded script contains an HTTP URL pointing to badwebsite.com with the path /xap_102b-AZ1/704e.php and a query parameter. Treat this as a lab indicator and do not access it outside an authorized environment.
File name	The script builds a Windows executable filename from the CommonApplicationData path; the visible decoded path in the supplied brief is associated with HSTH… .exe. Because the source PDF rendering splits part of the encoded blob, verify the exact filename against the lab's own decoder output before final submission.
Folder	CommonApplicationData, obtained using [System.Environment]::GetFolderPath("CommonApplicationData").
ShellExecute	The script calls the Document.Application.ShellExecute method through a dynamically created COM/object type, which is used to launch/open a file or application.
Do not execute the decoded script or visit the embedded URL. For the assignment, decode it offline and document the indicators only.	

3. Exercise 2 – Process Injection & Memory Injection Methods

The source asks what process injection is, which malware variants use it, and for at least four memory-injection methods. ṆfileciteÖturn1file0ÖL68-L74Ọ

3.1 What is process injection?

Process injection is a technique in which code or executable content is placed into the memory space of another process so that execution occurs under, or in the context of, that process. Attackers may use it for stealth, privilege use, defense evasion or access to another process's resources. Defenders look for unusual memory allocation, cross-process access, thread creation, image/module anomalies and parent-child/process-context inconsistencies.

3.2 Malware families / threat software associated with injection

The technique has been documented across many malware families and intrusion tools. Examples commonly discussed in defensive literature include Meterpreter, TrickBot, Emotet-era tooling, Cobalt Strike components and various credential-stealing or remote-access malware. The important analytical point is that injection is a technique, not a unique malware signature.

3.3 Four memory/process injection methods

Method	Description	Defensive indicators
DLL injection	A malicious DLL is caused to load into a target process. Common implementation patterns involve cross-process memory operations and a remote execution mechanism.	Unexpected DLL loads, remote-thread activity, unsigned modules, unusual parent/child context.
Process hollowing	A legitimate process is created or selected, its original image is replaced or its execution context is redirected, and attacker-controlled code is executed in the process context.	Suspicious process creation followed by image/memory inconsistencies, unbacked executable memory and mismatched process image characteristics.
Reflective DLL loading	A DLL is loaded from memory without relying on the normal Windows loader path in the conventional way.	Executable private memory, unusual PE structures in memory, suspicious thread start addresses and memory-only modules.
Thread/APC-based injection	Code is placed in a target process and execution is triggered through a thread or asynchronous procedure-call mechanism.	Cross-process memory writes, unusual thread start locations, APC activity and execution from non-module-backed memory.
Lab safety: demonstrate injection only in an isolated, authorized test environment. For a report, focus on detection artifacts rather than providing deployable injection code.		

4. Exercise 3 – Sysinternals & PE/Binary Analysis

The source specifically asks for at least three Sysinternals tools, screenshots, information obtained, execution analysis use, and whether each is static or dynamic. It also provides a PE-section/import figure for interpretation. ṆfileciteÖturn1file0ÖL75-L94Ọ

4.1 Recommended Sysinternals tools

Tool	Primary use	Information obtained	Analysis type
Sigcheck	File signature/version/hash and trust verification.	Digital signature status, publisher, version, hashes and other file metadata.	Primarily static.
Strings	Extract readable strings from binaries/files.	URLs, paths, registry strings, error messages, commands and embedded text that may provide analysis leads.	Static.
Process Explorer	Inspect running processes and loaded modules.	Process tree, executable path, command line, loaded DLLs, handles, signatures	Dynamic.

		and process relationships.	
Process Monitor (Procmon)	Observe real-time Windows activity.	Process, file-system, registry and network-related events with timestamps and filters.	Dynamic.

4.2 How an analyst uses the tools

- Sigcheck: establish file identity, hashes, publisher/signature information and whether the file appears trusted or unsigned.
- Strings: identify suspicious URLs, PowerShell commands, file paths, registry locations or other artifacts that can guide deeper analysis.
- Process Explorer: observe how a suspicious executable runs, what modules it loads, which process launched it and whether the binary is signed.
- Process Monitor: correlate execution with file, registry and process activity to build a timeline of what the sample attempted to do.

Tool screenshots should be taken from the authorized analysis VM and inserted in the spaces below.

Sigcheck – Lab Screenshot
Insert the Sigcheck screenshot captured in the isolated analysis VM.

Strings – Lab Screenshot
Insert the Strings screenshot captured in the isolated analysis VM.

Process Explorer – Lab Screenshot
Insert the Process Explorer screenshot captured in the isolated analysis VM.

Process Monitor – Lab Screenshot
Insert the Process Monitor screenshot captured in the isolated analysis VM.

4.3 Interpreting the PE figure supplied in the brief

The figure on page 4 shows a PE-file section table and an Imports tree. The visible sections include UPX0, UPX1 and an additional section, with entropy and MD5 columns. The imports include KERNEL32.DLL and MSVCRT-related entries. ṆfileciteÖturn1file0ÖL83-L94Ọ

Question	Answer
a. What do you see?	A Portable Executable analysis view showing target architecture, entry point, section names, virtual/raw sizes, entropy, hashes and imported libraries/functions.
b. What does the section mean?	PE sections are logical regions of an executable containing code, data and/or other loader-related information. Section metadata helps analysts understand layout and identify anomalies.
c. What does UPX mean?	UPX is an executable packer. UPX-packed sections such as UPX0/UPX1 are commonly associated with compressed/packed executable content.
d. What is entropy?	Entropy is a statistical measure of byte randomness. Higher

	entropy can be associated with compressed or encrypted/packed data, although entropy alone does not prove maliciousness.
e. What does the import section mean?	It lists external DLLs and functions that the executable expects to use. Imports provide behavioral clues, such as networking, process, file, registry or memory-related capabilities.
f. Kernel32.dll imports	The figure shows VirtualFree, ExitProcess, VirtualProtect, LoadLibraryA, VirtualAlloc and GetProcAddress. These are Windows API functions associated with memory management, dynamic library/function resolution and process termination; their presence is not by itself proof of malware.

5. Exercise 4 – Customer Malware Notification

The scenario states that Acme Incorporated experienced a phishing-led infection across ABC, CDE and FGH systems. Users received a malicious document, launched it, and the document downloaded Emotet from the Internet; the download succeeded and the systems were compromised. The assignment asks for a brief, customer-friendly notification, attack-flow explanation and recommended remediation. ṆfileciteÖturn1file0ÖL95-L125Ộ

5.1 Customer notification – executive version

Subject: Security Incident Notification – Phishing-Driven Malware Infection Detected

Dear Customer,

During our review of security events, we identified a phishing-based malware incident affecting systems ABC, CDE and FGH. The initial entry point was a malicious email received by the affected users. The users opened the malicious document, which then downloaded an Emotet malware payload from the Internet. Our analysis indicates that the download completed successfully and the affected systems were compromised.

Attack flow: Phishing email → malicious document opened → external malware download → Emotet execution/compromise → security events observed during investigation.

We recommend immediately isolating the affected systems, preserving relevant forensic evidence, resetting potentially exposed credentials, blocking identified malicious indicators, reviewing email and endpoint logs, and validating that no additional systems were compromised. The incident should remain under formal incident-response and evidence-preservation procedures until containment and eradication are confirmed.

Regards,
Security Operations / Incident Response Team

5.2 Information to include for the customer

Area	Information to communicate
What happened	Phishing email delivered a malicious document; the document was launched and downloaded malware.
Affected assets	ABC, CDE and FGH, as stated in the scenario.
Attack flow	Email → document execution → external download → malware compromise.
Threat	Emotet, as specified in the scenario.
Business impact	State only validated impact; do not speculate about data theft or lateral movement unless evidence confirms it.
Containment	Isolate affected endpoints, block indicators and restrict further execution/download paths.
Credential protection	Reset credentials that may have been exposed and review privileged accounts.
Investigation	Preserve evidence, review endpoint/email/network telemetry and scope the incident.
Recovery	Eradicate the malware, restore from trusted sources where required, patch affected systems and monitor for recurrence.
Lessons learned	Strengthen email filtering, attachment controls, endpoint detection, user awareness and incident-response playbooks.

6. Scenario-Based Questions

6.1 Scenario 1 – Protecting an organization during a critical attack

7. Activate the incident-response process and assign an incident commander.
8. Classify the incident and establish a communication bridge for security, infrastructure, application, legal/compliance and management stakeholders.
9. Contain affected systems using network isolation, endpoint controls and account/session revocation as appropriate.
10. Preserve forensic evidence before destructive remediation wherever practical.
11. Identify the initial access vector, affected assets, persistence, lateral movement and business impact using available telemetry.
12. Block confirmed indicators and close the exploited control gap.
13. Eradicate the threat, restore services from trusted sources and validate recovery.
14. Complete root-cause analysis, lessons learned, corrective actions and management reporting.

6.2 Scenario 2 – Repeated phishing from one domain

15. Collect representative emails and preserve full headers for analysis.
16. Validate whether the common domain is malicious, compromised or merely being impersonated/spoofed.
17. Block confirmed malicious sender/domain indicators at the email security gateway, DNS/security controls and endpoint layer as appropriate.
18. Search historical mailboxes for the same sender/domain and identify delivered, opened and clicked messages.
19. Quarantine matching messages and investigate users who interacted with them.
20. Reset credentials/session tokens when compromise is suspected.
21. Strengthen SPF, DKIM and DMARC controls for the organization's own domains and tune anti-phishing policies.
22. Run targeted user awareness and track repeat reporting/interaction trends.
23. Document indicators, actions, scope and lessons learned.

6.3 Scenario 3 – Red-team workflow for a new product

A defensible red-team workflow should be authorization-led, risk-based and evidence-driven.

24. Obtain written authorization, scope, rules of engagement, test windows and emergency contacts.
25. Perform asset discovery and architecture review for the product and supporting services.
26. Conduct threat modeling and identify high-value attack paths and trust boundaries.
27. Perform vulnerability assessment, secure configuration review, authentication/authorization testing, API/web testing, input validation testing and relevant client-side testing.
28. Use controlled exploitation only to validate findings within the approved scope and without unnecessary impact.
29. Document each finding with title, affected asset, description, evidence, severity rationale, attack preconditions, business/security impact and remediation.
30. Retest remediated findings and update the risk status.
31. Produce an executive summary plus a technical report and track residual risks through formal risk acceptance or remediation.

Important wording: No security assessment can guarantee that a product is completely “zero-day proof.” The objective is to reduce attack surface, identify known weaknesses, test realistic attack paths and establish a process for handling newly discovered vulnerabilities.

7. Evidence & Submission Checklist

Item	Status / action
Exercise 1 – cloud/data security answers	Completed in report.
Exercise 1 – forensic evidence handling	Completed in report.
Exercise 1 – encoded PowerShell analysis	Completed at indicator/behavior level; verify exact filename locally if required.
Encoded script screenshot	Insert screenshot of the source/decoder output from the

	authorized lab.
Exercise 2 – process injection	Completed.
Exercise 2 – four injection methods	Completed.
Exercise 3 – three+ Sysinternals tools	Completed; insert actual tool screenshots.
PE figure interpretation	Completed using the supplied page-4 figure.
Exercise 4 – customer notification	Completed.
Scenario 1/2/3 responses	Completed.
Final evidence review	Insert the lab screenshots and verify the recorded observations before final submission.

8. Conclusion

This capstone response combines governance, forensic integrity, malware analysis, endpoint telemetry, phishing response and red-team methodology. The strongest submission should pair the written answers with screenshots captured directly from the authorized lab environment and preserve clear evidence of the analyst's work.

Source note: The assignment brief is the controlling source for the questions, scenario wording and supplied PE figure; the response expands those questions into structured security-analysis answers. ṆfileciteÖturn1file0ÖL7-L20Ộ